

**UNIVERSIDAD AUTÓNOMA
METROPOLITANA**

Unidad Cuajimalpa

**División de Ciencias Naturales e Ingeniería
Licenciatura en Ingeniería en Computación**

**Punto de Venta Vinculado con un
Bot de Discord**

Proyecto Terminal

Autor:

Álvaro Arturo Ramírez Arriaga

Asesores:

Dr. José Netz Romero Durán

Mtra. Núñez Reyes Alba Rocío

Ciudad de México, Agosto 2025

Índice

1. Introducción	4
1.1. Planteamiento del Problema	5
2. Objetivos	6
2.1. Objetivo General	6
2.2. Objetivos Específicos	6
3. Estado del Arte	7
3.1. PayPal	7
3.2. Meet 6 Bot Discord	9
3.3. Tienda de Uniformes Escolares	12
3.4. Facebook Marketplace	14
3.5. AliExpress	15
4. Definición General del sistema	16
4.1. Propuesta	16
4.2. Tecnologías	17
4.2.1. PayPal	17
4.2.2. Discord	17
4.2.3. Node.js	18
4.2.4. React	18
4.2.5. MySQL	19
4.3. Requerimientos Funcionales	20
4.3.1. Página Web	20
4.3.2. Bot Discord	20
4.4. Requerimientos no Funcionales	21
4.4.1. Página Web	21
4.4.2. Bot Discord	21
5. Diseño del Sistema	22
5.1. Diagrama General	22
5.2. Diagrama Secuencial	23
5.3. Diagrama de la Base de Datos	25
6. Desarrollo del sistema	27
6.1. APIS	27
6.1.1. API de PayPal	27
6.1.2. API de Discord	28
6.2. Bot de Discord	29
6.2.1. Creación y Funcionamiento de Comandos	29
6.2.2. Notificación de Tickets Pendientes	30
6.2.3. Verificación de Propiedad de Ticket	30
6.3. Punto de Venta	30

7. Resultados	33
7.1. Interfaces del Punto de Venta	33
7.1.1. Interfaz de Inicio de Sesión por Discord	33
7.1.2. Interfaz de Página de Productos	34
7.1.3. Interfaz del Carrito y el Procesamiento de Tickets	35
7.2. funcionalidades del Bot	35
7.2.1. Creación de Comandos para Usuarios	35
7.2.2. Comprobación de Tickets Pendientes	36
7.2.3. Envío de Notificaciones por Canales Privados	37
8. Conclusiones	38
9. Referencias	39

1. Introducción

En la actualidad, la digitalización de los procesos comerciales ha transformado la manera en que las empresas y los consumidores interactúan. Las plataformas de mensajería instantánea y redes sociales, como Discord, han dejado de ser simples herramientas de comunicación para convertirse en espacios donde se pueden gestionar comunidades, ofrecer soporte y hasta realizar transacciones comerciales. Este proyecto surge de la necesidad de aprovechar estas plataformas modernas para mejorar la experiencia de compra y la gestión de ventas en pequeños negocios y emprendimientos.

El objetivo principal de este trabajo es el desarrollo de un sistema de punto de venta (POS) en línea que se integra de manera nativa con un bot de Discord. Esta integración permite centralizar la comunicación entre clientes y vendedores, automatizar notificaciones sobre compras y suscripciones, y facilitar la gestión de pedidos en tiempo real. El sistema está diseñado para que los usuarios puedan autenticarse utilizando su cuenta de Discord, navegar por un catálogo de productos, agregar artículos a su carrito, realizar pagos seguros mediante PayPal y recibir notificaciones automáticas sobre sus compras directamente en Discord.

El desarrollo del proyecto abarca varias áreas tecnológicas: el frontend está construido con React, proporcionando una interfaz moderna, responsiva y fácil de usar; el backend utiliza Node.js y Express para gestionar la lógica de negocio y la comunicación con la base de datos MySQL, donde se almacena toda la información relevante de usuarios, productos, carritos y tickets de compra. La integración con Discord se realiza mediante la librería Discord.js, que permite crear un bot capaz de interactuar con los usuarios, responder a comandos y enviar notificaciones personalizadas. Además, se implementa la autenticación OAuth2 de Discord para garantizar la seguridad y simplicidad en el acceso de los usuarios.

Uno de los aspectos más innovadores del sistema es la automatización de notificaciones: cada vez que un usuario realiza una compra, se genera un ticket en la base de datos con una bandera especial que indica que está pendiente de notificación. El bot de Discord revisa periódicamente estos tickets y, al encontrar nuevos, envía mensajes directos a los usuarios con los detalles de su compra, asegurando que la información llegue de manera rápida y directa, incluso si el usuario no revisa su correo electrónico.

El proyecto también aborda retos importantes, como la gestión segura de sesiones mediante cookies, la protección de datos sensibles, la integración de pagos electrónicos y el manejo de las limitaciones impuestas por la API de Discord en cuanto al envío de mensajes. Si bien el sistema está orientado principalmente a pequeñas empresas o emprendimientos con comunidades activas en Discord, sienta las bases para futuras ampliaciones y mejoras que podrían adaptarse a negocios de mayor escala.

1.1. Planteamiento del Problema

En la actualidad se ha visto un crecimiento en la preferencia de las personas por las redes sociales y plataformas de mensajería instantánea como Discord, que combina comunidad, mensajería y redes sociales. Actualmente, muchas tiendas envían actualizaciones importantes únicamente por correo, lo que resulta en que los mensajes se ignoren o queden en carpetas de spam.

Recientemente, las redes sociales han empezado a emplear mensajes automatizados o bots para agilizar las interacciones con los usuarios, evitando así que estos esperen a que una persona responda. Estos bots son software diseñado para realizar tareas específicas de forma repetida a una mayor velocidad que un ser humano; esto permite agilizar muchas tareas simples como enviar informes y/o mensajes.

Este proyecto planteó integrar un bot de Discord con una tienda en línea para centralizar la comunicación entre cliente y vendedor en la red social mencionada. Y así, abordar dos problemas clave: la dispersión de la comunicación y la falta de visibilidad de la información de compra.

El bot que se planteó para este proyecto pretendía realizar notificaciones en tiempo real sobre pagos, actualizaciones de pedidos y recordatorios de suscripciones, facilitando la visibilidad de la compra. Discord se ha convertido en un canal cotidiano para todo tipo de usuarios; para muchos, incluso mejora la experiencia respecto al correo electrónico, ofreciendo notificaciones instantáneas, interacción directa y personalización.

2. Objetivos

En este apartado se presentan los objetivos planteados para guiar el desarrollo del proyecto y proponer una solución tecnológica para la gestión de compras y suscripciones mediante un bot de Discord.

2.1. Objetivo General

Desarrollar un sistema que permita el almacenamiento y consulta de los datos de compra de un punto de venta a través de un bot de Discord.

2.2. Objetivos Específicos

Para este proyecto se tienen los siguientes objetivos específicos.

- 1.- Configurar una base de datos que almacene y gestione la información de compras y suscripciones.
- 2.- Integrar un bot de Discord que permita realizar consultas a la base de datos.
- 3.- Desarrollar un sistema de notificaciones automatizadas.
- 4.- Evaluar las funcionalidades del sistema.
- 5.- Implementar un punto de venta.

3. Estado del Arte

Analizaremos sitios que cuenten con características o funcionalidades similares a las que usaremos para el proyecto que se va a desarrollar: las herramientas a analizar serán PayPal (forma de pago), bot de Discord MEE6, proyecto Tienda Virtual, Uniformes Escolares y Facebook Market.

3.1. PayPal

El método de pago de PayPal se basa principalmente en una implementación a través de su API, más que en el diseño de una interfaz propia. Esta API requiere información como la solicitud de pago, el precio del producto y la cuenta de PayPal a la que se debe depositar el dinero una vez que se complete la transacción. Por lo tanto, la única tarea que debe implementar el programador es la integración de la API y el botón para invocar el proceso de pago de PayPal.

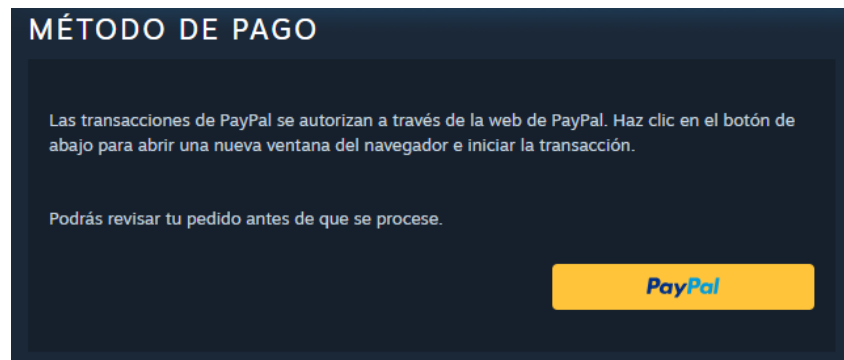


Figura 1: Botón de método de pago PayPal. [1]

El botón de la figura 1 no es más que una conexión de la página web, la sección de pago con el verdadero gestor de la transacción, como es PayPal, mediante la API.

Una vez llamada la API de PayPal, esta nos llevará a una ventana emergente (figura 2) donde se nos pedirá ingresar los datos de la cuenta de la plataforma PayPal para poder continuar con el cobro del servicio. Esta interfaz ya no es necesario crearla.

Inicie sesión en su cuenta de PayPal - Personal: Microsoft Edge

https://www.paypal.com/checkoutnow?atomic-event-state=eyJkb21haW4...

Pagar con PayPal

Ingresa su dirección de correo electrónico para empezar.

Correo electrónico o número de celular

[¿Olvidó su correo electrónico?](#)

Siguiendo

Abrir una cuenta

[Cancelar y volver a www.steampowered.com](#)

[Español](#) | [English](#)

[Contactar](#) [Privacidad](#) [Acuerdos legales](#) [En todo el mundo](#)

Figura 2: Formulario de inicio de sesión de PayPal. [1]

La figura 2 muestra la interfaz para la autenticación desde la API de PayPal, donde el usuario debe autenticarse con su cuenta de PayPal para acceder al sistema. Se utiliza el sistema OAuth2 de PayPal para este propósito.

El pago entraría casi de inmediato, solo faltaría confirmar el pago para que se realice el cobro del servicio y finalizaría así el cobro por vía PayPal.

3.2. Meet 6 Bot Discord

El bot que Meet 6 cuenta con muchas características interesantes para nosotros, pero en este proyecto nos centraremos en la posibilidad de enviar notificaciones de recordatorios por un canal en específico o por mensaje directo al usuario.

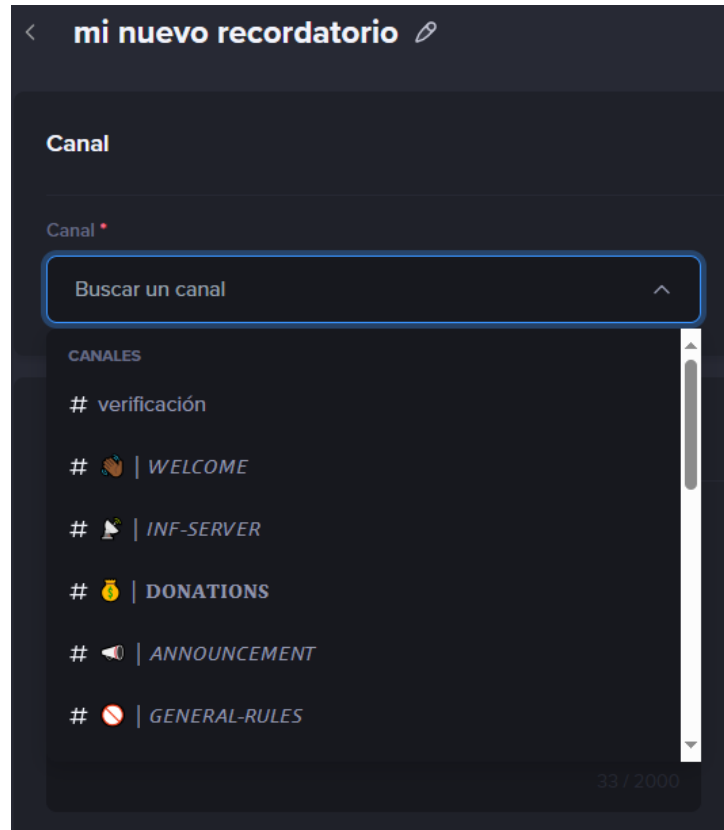


Figura 3: Selección de canal para recordatorio. [2]

En la ventana de la figura 3 podremos seleccionar en qué canal queremos que se genere el recordatorio que vamos a colocar. Esto es útil para aplicar recordatorios para un grupo de personas con roles en específico dentro del servidor de Discord. De esta forma, solo llegarán los recordatorios a las personas que queremos que reciban estos mensajes. También podemos poner un auto rol para que ellos decidan si quieren recibir los recordatorios o no, algo similar a cuando te pregunta una empresa si quieres recibir correos de promociones.

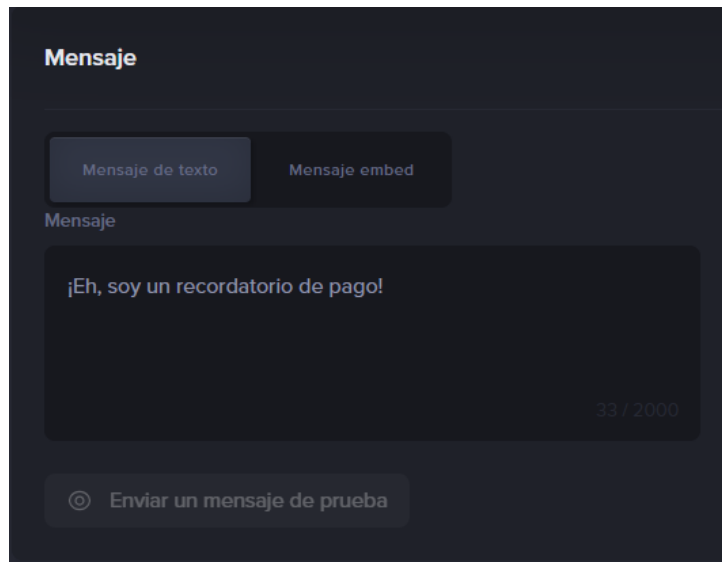


Figura 4: Ingreso del mensaje de recordatorio. [2]

La interfaz de la figura 4 permite ingresar el mensaje que deseamos que los usuarios reciban como recordatorio. En este caso, el mensaje consistirá únicamente en texto, aunque también es posible agregar una imagen para que el bot la publique automáticamente junto con el texto. De esta forma, se puede generar una publicidad mucho más eficaz para los artículos y el público objetivo al que va dirigida.

Además, al tener la posibilidad de enviar mensajes directos a los usuarios, podremos notificarles sobre próximas fechas de pago para alguna suscripción activa, o incluso ofrecerles descuentos personalizados.

Recordatorio

Único Múltiples

A las 18:00

Zona horaria UTC

Día de la semana

✓ Mon ✓ Tue ✓ Wed ✓ Thu ✓ Fri ✓ Sat ✓ Sun

Figura 5: Configuración de recordatorios automáticos. [2]

La función de recordatorio se puede configurar de forma personalizada, como se muestra en la figura 5. Para que el recordatorio se genere automáticamente a intervalos de tiempo específicos, es necesario indicar el día y la hora. Esta funcionalidad es útil para evitar tener que crear el recordatorio de forma manual cada vez que se desee generar uno similar. En nuestro caso, tomará esta información de una base de datos, donde verificará si es necesario aplicar el recordatorio. En caso de tratarse de una mensualidad, recordará al usuario con 5 días de anticipación, revisando la fecha del siguiente cobro.

3.3. Tienda de Uniformes Escolares

En este proyecto desarrollado en la universidad Oberta de Catalunya se realizó una tienda online de uniformes escolares, del cual tomaremos algunas utilidades de este proyecto como referencia.



Figura 6: Listado de artículos de la tienda de uniformes. [3]

Como podemos ver en la figura 6, en la interfaz de compra únicamente hace falta dar clic sobre 'añadir al carrito' para poder adquirirlo; también se ve el precio del producto y su nombre.

Del lado izquierdo también se puede apreciar el filtro que se tiene para los productos, de esta forma se tiene una mejor organización sobre lo que está observando el usuario comprador en este momento. Al lado derecho se muestran los productos que han sido añadidos al carrito, así como el precio final de la compra.

En la figura 7 podemos ver qué pasa al darle clic a comprar, ahora nos manda a la página de pago donde saldrán los datos del comprador, en este caso se ve cómo el pago se carga automáticamente a la mensualidad del colegio, en nuestro caso habrá un método de pago para realizar el pago.



Uniforme Escolar

Categorías

- ABRIGO
- CALZADO
- DEPORTIVO
- NIÑA
- NIÑO

 Mis pedidos

Detalle de pedido

Nombre: Pruebas
Dirección: Calle de Pruebas
Teléfono: 916458752
Email: correo@uoc.edu
NIF: 53038956C
CCC: 123456789123456 (*)

(*) El importe total se cargará en la cuenta indicada junto con el próximo recibo mensual del colegio

Unidades.	Artículo	Precio
1	ABRIGO UNIFORME	59,9€
1	CALCETIN	5,95€
1	PANTALON CORTO SPORT	19,9€
Total:		85,75€

<< Atrás Confirmar >>

Figura 7: Resumen de compra y datos del comprador. [3]

Para nuestro proyecto será importante colocar una sección de productos para seleccionarlos y, después de seleccionarlos, colocar una descripción y más imágenes para que el comprador pueda ver y estar más seguro de que se está comprando, al igual que un desglose de su compra total.

3.4. Facebook Marketplace

Facebook Market también es otra de las plataformas más utilizadas para la compra y venta de productos, por esta razón es una buena referencia para el proyecto.

En la figura 8 podemos ver cómo se muestra la información del vendedor, detalles del producto y algunas especificaciones básicas; no es tan extensa la descripción como en Amazon. Sin embargo, nos da la posibilidad de enviarle un mensaje al comprador para poder preguntarle a él de una forma más directa y personalizada.

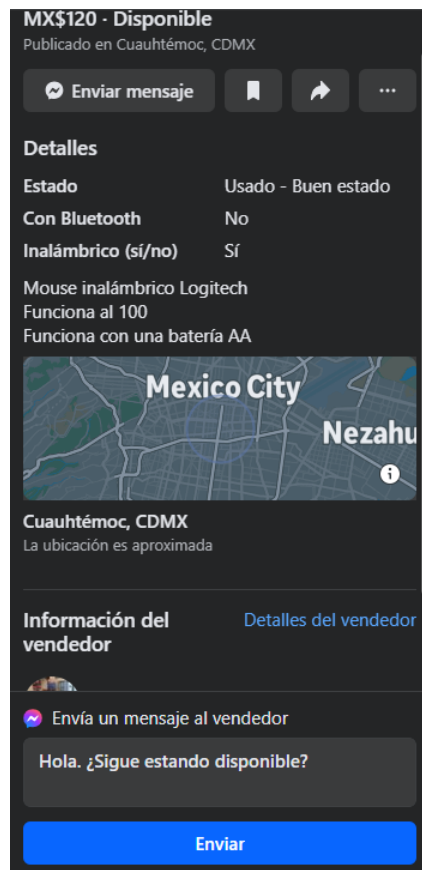


Figura 8: Información del producto y vendedor en Marketplace. [4]

La figura 8 muestra la sección informativa donde se puede ver el detalle del producto, así como la opción de contactar al vendedor.

Si bien esta plataforma está orientada a que terceros vendan sus productos en su tienda, son buenas referencias para este proyecto, al poder tener un trato directo con el vendedor.

Este proyecto tomará la idea de poder mandarle mensajes directamente al vendedor, en este caso, el administrador de la tienda, para algunas compras que requieran de más pasos y no solo vender el producto.

3.5. AliExpress

Algunas tiendas en línea ya utilizan actualmente notificaciones por medio de alguna red social; este es el caso de AliExpress, donde reenvían información básica de tu compra y links para visualizar la información completa de la compra en una página web. Si bien no es el ticket, sí facilita poder verlo sin necesidad de entrar al correo electrónico.

La figura 9 ejemplifica una notificación simplificada que recibiría el usuario tras realizar una compra.

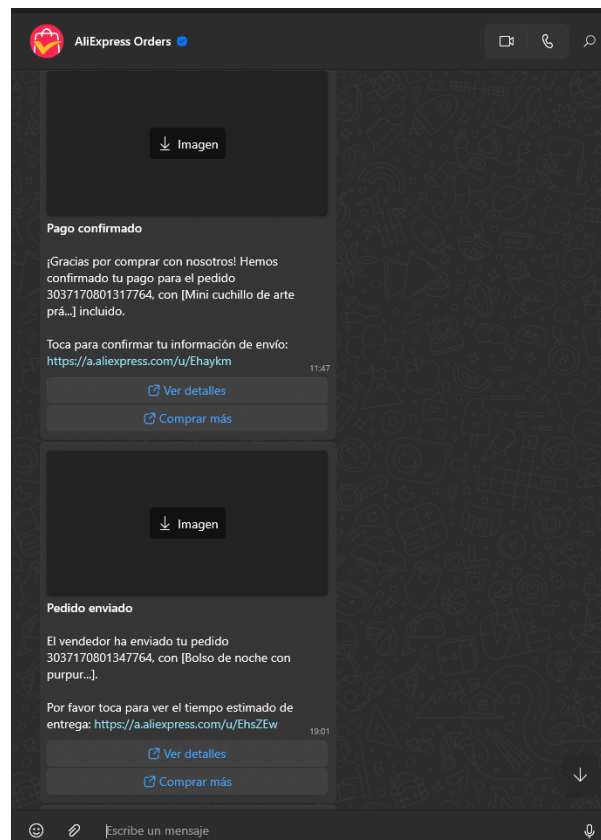


Figura 9: Notificación de compra reenviada (AliExpress). [5]

Este proyecto busca poder realizar notificaciones mediante una red social como es Discord para centralizar la forma de encontrar la información de compra y contactar con la tienda o persona encargada de esta de una forma mucho más fácil, utilizando únicamente Discord.

4. Definición General del sistema

4.1. Propuesta

Este proyecto propone el desarrollo de un sistema integrado con un bot de Discord para una tienda en línea, con el fin de centralizar la comunicación y optimizar la gestión de compras y suscripciones. A diferencia de otras soluciones existentes, la propuesta aprovecha la versatilidad y popularidad de Discord como plataforma para ofrecer notificaciones en tiempo real sobre pagos, actualizaciones de pedidos y recordatorios, mejorando la visibilidad y la interacción directa con los usuarios.

El sistema incluye el diseño y desarrollo de una base de datos robusta para almacenar y consultar información relevante, la integración del bot para realizar consultas y notificaciones automatizadas, y una interfaz administrativa que facilita la gestión de artículos, compras y suscripciones. Así, se busca proporcionar una solución tecnológica adaptada a las necesidades específicas de la tienda, que combine eficiencia, usabilidad y aprovechamiento de plataformas de comunicación actuales.

4.2. Tecnologías

Las tecnologías para este proyecto serán API PayPal, API Discord, MYSQL, PHP, Node.js, React. Se explicará cada una de las tecnologías para definir qué funcionalidad van a desempeñar en el proyecto propuesto.

4.2.1. PayPal

PayPal es una de las plataformas más populares para pagos en línea, conocida por su facilidad de uso, seguridad y aceptación internacional. Para este proyecto, se ha elegido PayPal debido a su amplia adopción por los usuarios y su capacidad para gestionar pagos transfronterizos de manera eficiente, ofreciendo una ventaja frente a competidores como Mercado Pago.

La integración de PayPal en una tienda en línea se realiza principalmente mediante su API. Esta API permite establecer la comunicación entre la tienda y los servidores de PayPal para gestionar las transacciones de manera segura. Los pasos básicos para utilizar PayPal como método de pago son:

4.2.2. Discord

Discord es una plataforma de comunicación ampliamente utilizada para la creación y gestión de comunidades temáticas. Originalmente orientada hacia los jugadores, Discord ha evolucionado para ser una herramienta versátil que permite administrar grandes grupos de usuarios gracias a su estructura basada en servidores. En estos servidores, se pueden asignar roles, gestionar permisos y utilizar bots para automatizar diversas tareas, convirtiéndola en una solución ideal para centralizar la interacción entre usuarios y servicios.

Discord será una herramienta central en este proyecto, gracias a su capacidad para gestionar usuarios y automatizar notificaciones. Se aprovecharán tanto su API como las características de inicio de sesión para integrar la tienda en línea con la plataforma.

Se utilizará la OAuth2 API de Discord para implementar un sistema de autenticación seguro y sencillo. Esto permitirá a los usuarios iniciar sesión en la tienda en línea con su cuenta de Discord, sin necesidad de crear un nuevo usuario.

4.2.3. Node.js

Node.js es un entorno de ejecución que permite ejecutar JavaScript fuera del navegador. Basado en el motor V8 de Google Chrome, Node.js es extremadamente rápido y eficiente, especialmente para aplicaciones que requieren manejo en tiempo real.

Node.js es ideal para aplicaciones que necesitan manejar múltiples solicitudes simultáneamente, como un bot de Discord que responde a eventos en tiempo real. También tiene una muy buena estabilidad, por lo cual aunque haya muchas solicitudes simultáneas conserva un buen rendimiento.

Para este proyecto utilizaremos Node.js junto con una biblioteca llamada discord.js que simplifica la interacción con la API de Discord, la cual nos permitirá programar un bot que pueda enviar y recibir mensajes.

Esta api nos dará la posibilidad de asignar o modificar roles de usuarios automáticamente según ciertas condiciones. También nos dará la facilidad de que el bot pueda enviar mensajes directos, incrustaciones (embeds) con diseño visual atractivo, y notificaciones personalizadas.

Usando Discord.js, el bot enviará mensajes automáticos a los usuarios tras completar una compra, actualizando estados de pedidos o recordando renovaciones de suscripciones.

4.2.4. React

React es una biblioteca de JavaScript desarrollada por Facebook, diseñada para construir interfaces de usuario (UI) modernas, interactivas y dinámicas. Se enfoca principalmente en la creación de aplicaciones a través de componentes reutilizables, lo que permite dividir la interfaz en pequeñas piezas independientes y manejables. Esto mejora la organización y el mantenimiento del código; también facilita la reutilización de estos componentes en diferentes partes de la aplicación. En este proyecto, un componente podría encargarse de mostrar los productos de la tienda, otro del carrito de compras y otro de las notificaciones de compra, optimizando así el desarrollo.

Una de las características más destacadas de React es el uso del Virtual DOM. Este mecanismo crea una representación en memoria del DOM real, lo que permite que React identifique y actualice solo las partes necesarias de la interfaz en lugar de renderizarla por completo cada vez que hay un cambio. Esto mejora significativamente el rendimiento de la aplicación, especialmente en escenarios donde las actualizaciones en tiempo real son frecuentes, como al agregar productos al carrito o actualizar el estado de un pedido.

React fue elegido para este proyecto debido a su escalabilidad y facilidad de mantenimiento. React ofrece un equilibrio ideal entre flexibilidad y robustez, lo que lo convierte en una herramienta esencial para construir una tienda en línea moderna que integre funcionalidades avanzadas como notificaciones en tiempo real y bots automatizados.

4.2.5. MySQL

MySQL es un sistema de gestión de bases de datos relacional, se utiliza para almacenar, organizar y administrar grandes volúmenes de información en aplicaciones de software. Este sistema utiliza el lenguaje SQL (Structured Query Language) para interactuar con los datos, permitiendo realizar operaciones como consultas, inserciones, actualizaciones y eliminaciones de manera eficiente y estructurada.

Una de las principales ventajas de MySQL es su capacidad para manejar grandes cantidades de datos de manera rápida y eficiente. Gracias a su diseño optimizado, MySQL permite realizar consultas complejas que filtran y procesan información específica en cuestión de milisegundos. Esto es crucial en aplicaciones como tiendas en línea, donde se manejan continuamente datos relacionados con usuarios, productos, pedidos y transacciones.

Otra característica importante de MySQL es su enfoque en la integridad de los datos y la seguridad. A través de herramientas como Prepared Statements (consultas preparadas), es posible proteger la base de datos contra ataques comunes como la inyección SQL, garantizando que las consultas se ejecuten de forma segura y controlada. Esta funcionalidad es esencial en proyectos que manejan información confidencial, como detalles de usuarios y transacciones económicas.

En este proyecto, MySQL se utilizará para gestionar tablas que almacenen información esencial como detalles de los usuarios, productos, pedidos y estados de pago. Además, permitirá realizar operaciones frecuentes, como buscar productos según filtros personalizados, actualizar el estado de un pedido en real tiempo o registrar nuevas transacciones.

4.3. Requerimientos Funcionales

A continuación, se presentan los requerimientos funcionales del proyecto, los cuales describen las acciones y comportamientos que el sistema deberá cumplir para satisfacer las necesidades del usuario y alcanzar los objetivos planteados. Estos requerimientos definen qué funciones específicas realizará el sistema, incluyendo la interacción con el bot de Discord, la gestión de pedidos y la comunicación con los clientes.

4.3.1. Página Web

- Permitir a los usuarios navegar por los productos disponibles en la tienda virtual.
- Permitir a los usuarios autenticarse mediante Discord (OAuth2).
- Permitir a los usuarios agregar productos al carrito de compras.
- Permitir a los usuarios modificar la cantidad de productos en el carrito y eliminar productos del mismo.
- Permitir a los usuarios realizar compras y generar tickets de compra usando PayPal.
- Mostrar notificaciones de éxito o error en las operaciones principales (login, agregar al carrito, compra).
- Permitir a los usuarios ver su perfil y cerrar sesión.
- Mostrar información relevante de los productos (nombre, imagen, precio, categoría).
- Permitir la navegación entre diferentes páginas (productos, carrito, perfil).
- Integración con un servidor de Discord (botón de invitación y QR).

4.3.2. Bot Discord

- Permitir la integración con el servidor de Discord de la tienda virtual.
- Responder a comandos básicos de los usuarios `/productos`.
- Notificar en canales específicos cuando se realiza una compra o se genera un ticket en la tienda.
- Permitir la gestión de roles o permisos para usuarios que compran productos.
- Permitir la consulta de información de productos o tickets desde Discord (si aplica).

4.4. Requerimientos no Funcionales

En esta sección se detallan los requerimientos no funcionales del proyecto, los cuales establecen los criterios de calidad que el sistema debe cumplir.

4.4.1. Página Web

- La aplicación debe ser responsiva y funcionar correctamente en dispositivos móviles y de escritorio.
- El sistema debe mantener la sesión del usuario mediante cookies seguras.
- El backend debe manejar la autenticación de manera segura.
- El sistema debe manejar errores y mostrar mensajes claros al usuario.
- El sistema debe ser escalable para permitir la integración de nuevas funcionalidades (por ejemplo, más métodos de pago, más categorías).
- El sistema debe proteger los datos sensibles y cumplir con buenas prácticas de seguridad.
- El backend debe soportar concurrencia y múltiples usuarios simultáneos.
- El sistema debe permitir la integración con servicios externos como Discord y PayPal.

4.4.2. Bot Discord

- El bot debe estar disponible 24/7 y manejar desconexiones de manera automática.
- El código debe estar organizado y documentado para facilitar el mantenimiento.
- El bot debe manejar errores y excepciones sin interrumpir su funcionamiento.
- El bot debe proteger los tokens y credenciales de acceso.
- El bot debe ser escalable para permitir la adición de nuevos comandos o integraciones.

5. Diseño del Sistema

5.1. Diagrama General

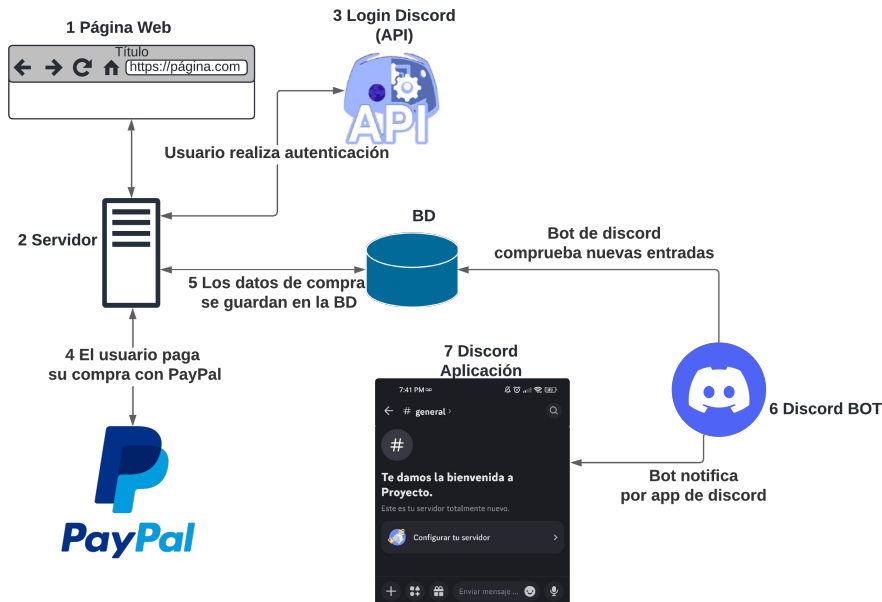


Figura 10: Diagrama general del sistema

- 1.- El usuario podrá consultar los artículos a la venta en el sitio web.
- 2.- El servidor será el encargado de procesar todas las solicitudes que conectarán el front-end con toda la lógica.
- 3.- El sistema autenticará al usuario mediante su cuenta de Discord.
- 4.- Una vez teniendo artículos en el carrito de compra, el usuario podrá comprarlos y pagarlos mediante PayPal; deberá realizar una autenticación dentro de PayPal para poder acceder a su cuenta y realizar el pago.
- 5.- En la base de datos se guardarán todos los datos necesarios para autenticación, los datos de compra, y los artículos que se han colocado a la venta.
- 6.- El bot de Discord hará una función administrativa, esta será avisarle al administrador y al usuario comprador sobre su compra o próximas suscripciones. Estos avisos serán personales para cada usuario.
- 7.- Mediante Discord, el usuario recibirá las notificaciones que le enviará el bot; el usuario requiere un perfil dentro de Discord.

5.2. Diagrama Secuencial

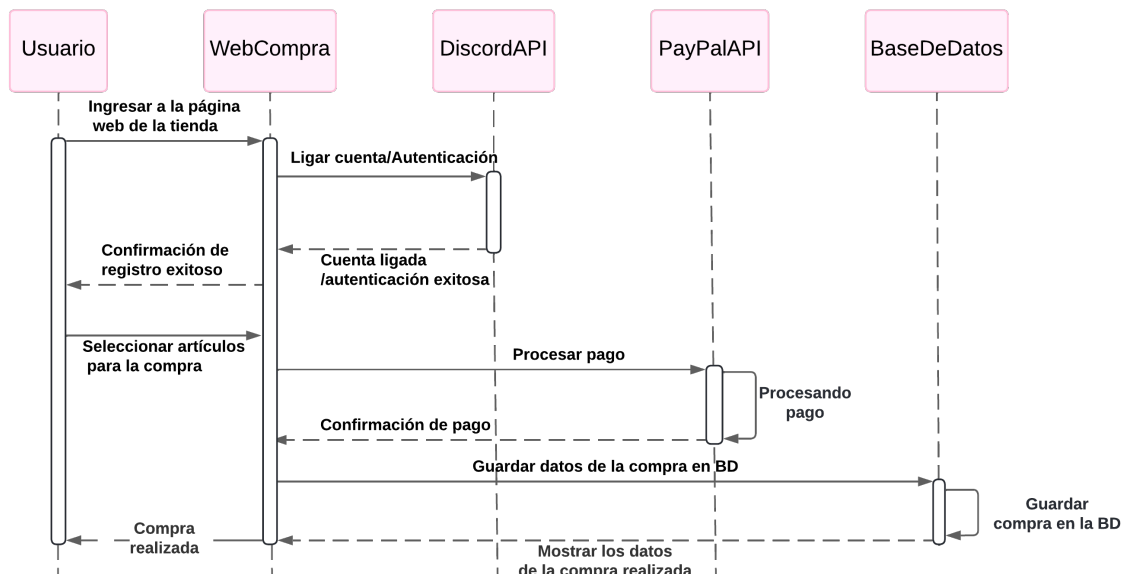


Figura 11: Diagrama secuencial de autenticación y compra

- 1.- El usuario deberá iniciar sesión, autenticarse en la página web para poder realizar compras.
- 2.- Una vez seleccionado el artículo de compra, se mandará al usuario a realizar su compra; se usará PayPal para realizar la validación de compra.
- 3.- La compra realizada se registrará en la base de datos.
- 4.- Se le mostrará al usuario un mensaje de compra exitosa.

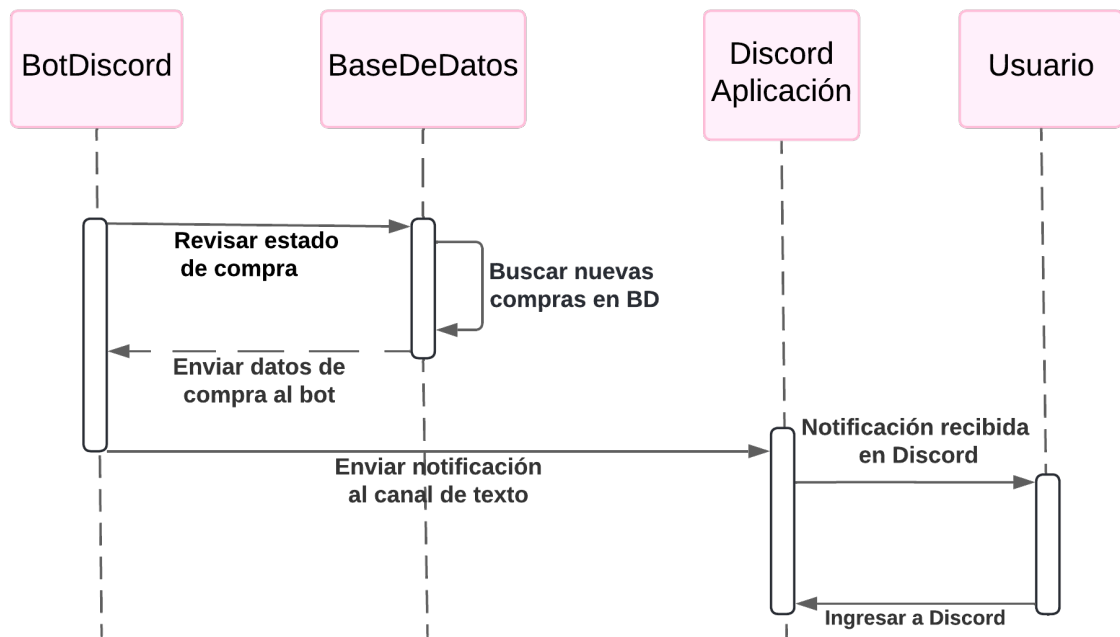


Figura 12: Diagrama secuencial de notificación por bot

- 1.- El bot de Discord comprobará si hay nuevas entradas en la BD.
- 2.- El bot enviará los datos de compra vía Discord.
- 3.- El usuario comprador podrá ver los datos de su compra en el mensaje dejado por el bot.

5.3. Diagrama de la Base de Datos

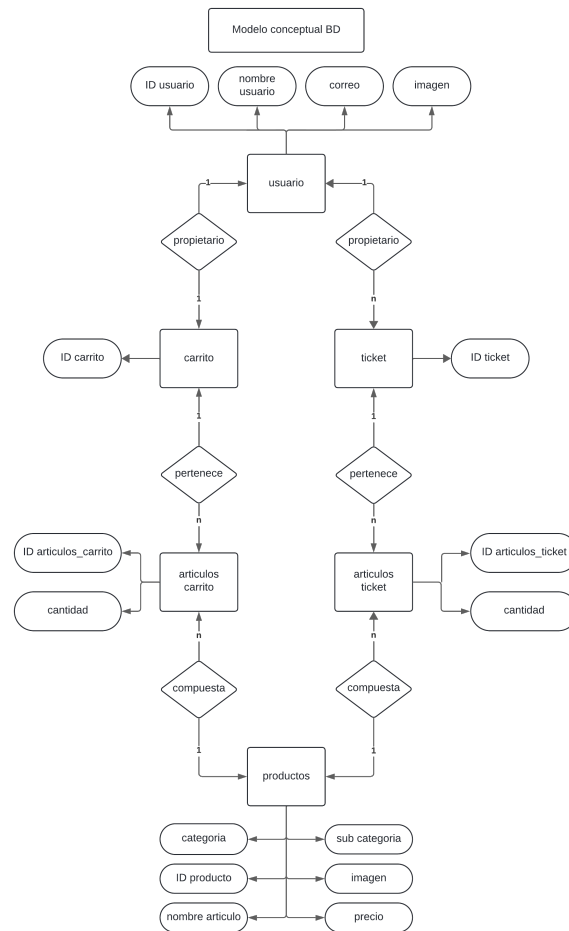


Figura 13: Modelo conceptual de la base de datos

El diagrama de la figura 13 comienza con la entidad usuario, que contiene información básica como ID, nombre, correo e imagen. Cada usuario puede ser propietario de un carrito y puede tener muchos tickets, lo que implica que un mismo usuario puede realizar varias compras, pero solo tener un carrito activo a la vez.

El carrito está relacionado con el usuario mediante una relación de propiedad uno a uno, y contiene múltiples artículos del carrito, es decir, productos que el usuario ha agregado antes de comprar. Estos artículos tienen una cantidad y se relacionan con la entidad productos, lo cual indica qué producto específico se añadió y en qué cantidad.

Una vez que el usuario realiza una compra, se genera un ticket, también asociado al usuario, pero en una relación de uno a muchos (un usuario puede tener muchos tickets). El ticket contiene artículos del ticket, similares a los del carrito, con su respectiva cantidad y referencia al producto adquirido.

La entidad productos es el catálogo general, donde se guarda información de cada artículo como su ID, nombre, categoría, subcategoría, imagen y precio. Esta entidad es común tanto para los artículos del carrito como para los del ticket, ya que los productos

son los mismos, solo cambian de estado según estén en proceso de compra (carrito) o ya comprados (ticket).

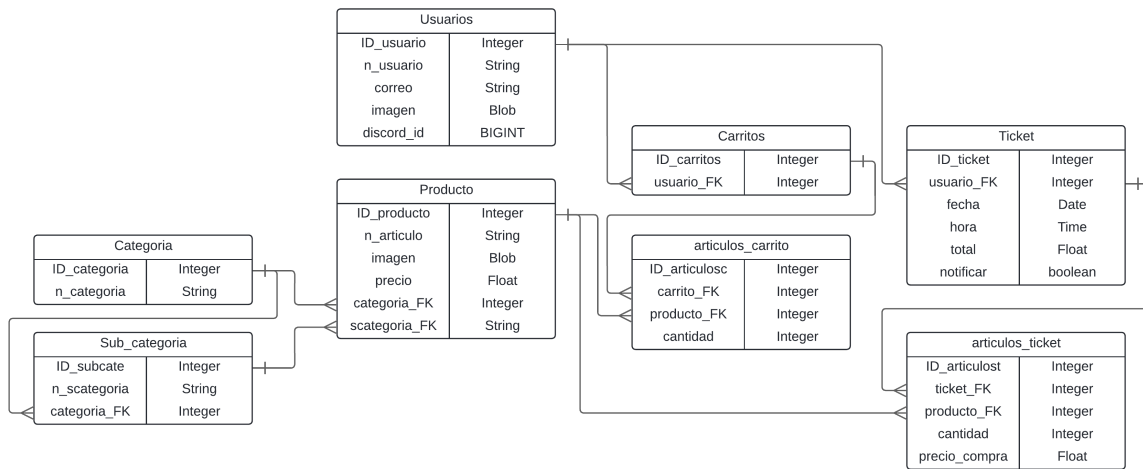


Figura 14: Diagrama lógico de la base de datos

En el diagrama de la figura 14 en base de datos, la entidad Usuarios almacena la información del cliente como ID, nombre, correo e imagen. Cada usuario puede tener un carrito (relación uno a uno) y también puede generar tickets de compra (relación uno a muchos).

El carrito está relacionado con el usuario mediante la clave foránea usuario_FK y contiene productos a través de la tabla articulos_carrito, donde se guarda qué producto fue agregado producto_FK y en qué cantidad.

Cuando se realiza una compra, se genera un ticket que también pertenece a un usuario (usuario_FK) y contiene información como fecha, hora, total de la compra y si se notificó. Los productos comprados se guardan en la tabla articulos_ticket, con su cantidad y el precio al momento de la compra (precio_compra), todo enlazado al ticket (ticket_FK) y al producto (producto_FK).

La entidad Producto almacena todos los artículos disponibles: su ID, nombre, imagen, precio y dos claves foráneas que lo conectan con su categoría y subcategoría (categoria_FK y scategoria_FK). Las categorías están organizadas en dos niveles, mediante las tablas Categoria y Sub_categoria, esta última relacionada con la primera a través de categoria_FK.

6. Desarrollo del sistema

6.1. APIS

En esta sección se describen las APIs que formarán parte del sistema, tanto internas como externas. Las APIs (Interfaces de Programación de Aplicaciones) permiten la comunicación entre los diferentes componentes del sistema, como la tienda en línea y el bot de Discord, facilitando el intercambio de datos de forma estructurada y segura. A través de estas interfaces, el sistema podrá recibir eventos, enviar notificaciones y ejecutar funcionalidades clave que permiten la integración fluida entre la plataforma de e-commerce y Discord.

6.1.1. API de PayPal

1.- Creación de una cuenta de PayPal para negocios: Debemos registrar una cuenta empresarial en el sitio web de PayPal. Esto permitirá vincular la cuenta a la tienda y recibir pagos directamente en ella.

2.- Configuración en el portal de desarrolladores de PayPal: Acceder al portal de desarrolladores de PayPal (<https://developer.paypal.com>) para obtener credenciales de API, como: - Client ID. - Secret Key. Estas credenciales son necesarias para autenticar las solicitudes desde la tienda hacia los servidores de PayPal.

3.- Integración de la API en la tienda: Implementar la API de PayPal en el back-end de la tienda. Esto implica: Configurar la solicitud de pago: indicar el monto de la compra, la moneda y los detalles del destinatario (cuenta de PayPal del vendedor). Gestionar la respuesta de PayPal: verificar el estado del pago (éxito o error) y actualizar la información en la base de datos de la tienda.

4.- Incorporación de un botón de pago: En el front-end, se utiliza el botón de PayPal, el cual es generado dinámicamente por la API o configurado manualmente con el código proporcionado por PayPal. Este botón redirige a los usuarios a la plataforma de PayPal para completar la transacción.

5.- Redirección tras el pago Una vez completado el pago, el usuario es redirigido de regreso al sitio web de la tienda, donde se muestra una confirmación del pedido o un mensaje de error si la transacción falló.

Configuración

1. Crea una app en [PayPal Developer](<https://developer.paypal.com/>).
2. Obtén tu 'Client ID' y 'Client Secret' de sandbox.
3. Configura las credenciales en el backend:

estos coandos se recommenda colocarse en un archivo .env

```
PAYPAL_CLIENT_ID=...  
PAYPAL_CLIENT_SECRET=...
```

En el back-end

Código 1: Captura una orden de PayPal y genera el ticket

```
1 POST /api/paypal/capture-order
2 Body: { orderId: string, total: number }
```

En el front-end

Código 2: Al completar el pago con PayPal, enviar el orderId y total al backend

```
1 axios.post(`${process.env.REACT_APP_BACKEND_URL}/api/paypal/capture-
  order`, {
2   orderId,
3   total
4 }, { withCredentials: true })
5 .then(res => {
6   // Mostrar mensaje de exito y ticket
7 });
```

6.1.2. API de Discord

La API de Discord permite a los usuarios autenticarse en la tienda usando su cuenta de Discord mediante OAuth2. Al autenticarse:

- El usuario es redirigido a la página de autorización de Discord.
- Tras autorizar, Discord redirige al backend con un código temporal.
- El backend intercambia ese código por un token y obtiene los datos del usuario desde Discord.
- Si el usuario no existe en la base de datos, se crea automáticamente.
- Se genera una cookie de sesión ('user') con los datos básicos del usuario (id, username (nombre de usuario), avatar, email).
- El frontend puede leer esta cookie (vía endpoint '/auth/user') para mostrar el perfil y mantener la sesión.

Configuración:

Se realizó un registro de la aplicación en el [Portal de Desarrolladores de Discord] (<https://discord.com/developers/applications>) y obtener:

```
1 DISCORD_CLIENT_ID
2 DISCORD_CLIENT_SECRET
3 DISCORD_REDIRECT_URI (ejemplo: 'http://localhost:5000/auth/discord/callback')
```

Código 3: Se agregan estos valores al archivo '.env' del backend:

```
1 DISCORD_CLIENT_ID=...
2 DISCORD_CLIENT_SECRET=...
3 DISCORD_REDIRECT_URI=http://localhost:5000/auth/discord/callback
4 FRONTEND_URL=http://localhost:3000
```

Flujo y rutas principales de la API.

1. Iniciar login con Discord GET /auth/login

- Redirige al usuario a la página de autorización de Discord.
- Se guarda un 'state' aleatorio en una cookie para prevenir ataques CSRF.

2. Callback de Discord OAuth2

GET /auth/discord/callback

- Discord redirige a esta ruta tras la autorización.
- El backend valida el 'state' o estado y obtiene el token de acceso.
- Se consulta la API de Discord para obtener los datos del usuario.
- Se guarda/actualiza el usuario en la base de datos.
- Se crea una cookie 'user' con los datos básicos del usuario.
- Redirige al front-end de la tienda virtual.

3. Obtener usuario autenticado

GET /auth/user

- Devuelve los datos del usuario autenticado leyendo la cookie 'user'.
- Si no hay cookie, responde 401.

4. Logout

POST /auth/logout

- Elimina la cookie 'user' y termina la sesión.

Código 4: La cookie que se genera para guardar los datos del usuario 'user' contiene un JSON con:

```
1 {  
2   "id": "discord_id",  
3   "username": "nombre_usuario",  
4   "avatar": "url_avatar_discord",  
5   "email": "correo@ejemplo.com"  
6 }
```

Esta cookie es usada por el front-end para mostrar el perfil y por el back-end para identificar al usuario en operaciones como agregar al carrito o comprar.

6.2. Bot de Discord

El bot de Discord desarrollado para este proyecto utiliza la librería `discord.js` en Node.js, permitiendo la interacción directa con la API de Discord y la integración con la base de datos del sistema. A continuación, se describe su funcionamiento y se muestran fragmentos de código relevantes.

6.2.1. Creación y Funcionamiento de Comandos

En Discord, los comandos se definen como funciones que pueden ser invocadas por los usuarios mediante mensajes que comienzan con un prefijo o mediante comandos slash (/). En este proyecto, los comandos se registran al iniciar el bot y se manejan mediante eventos. Un ejemplo básico de la creación de un comando sería:

Código 5: Registro de un comando slash

```
1 client.on('ready', async () => {
2   const data = [{
3     name: 'tickets',
4     description: 'Consulta tus tickets de compra'
5   }];
6   await client.application.commands.set(data);
7 });
```

Cuando un usuario ejecuta el comando `/tickets`, el bot identifica al usuario mediante su ID de Discord y consulta la base de datos para obtener los tickets asociados a ese usuario.

6.2.2. Notificación de Tickets Pendientes

El bot revisa periódicamente la base de datos para encontrar tickets que tengan un campo booleano `notificado` con valor 1. Solo estos tickets serán notificados al usuario correspondiente. Una vez enviada la notificación, el bot actualiza el campo a 0, asegurando que, incluso si el bot se reinicia o falla, no se repitan notificaciones de tickets ya procesados.

Código 6: Notificación y actualización de tickets

```
1 const tickets = await db.query('SELECT * FROM tickets WHERE notificado
   = 1');
2 for (const ticket of tickets) {
3   const user = await client.users.fetch(ticket.discord_id);
4   await user.send('Tienes un nuevo ticket: #${ticket.id}');
5   await db.query('UPDATE tickets SET notificado = 0 WHERE id = ?', [
6     ticket.id]);
7 }
```

6.2.3. Verificación de Propiedad de Ticket

Cuando un usuario ejecuta un comando para consultar detalles de un ticket, el bot verifica que el ticket pertenezca al usuario que ejecutó el comando. Esto se realiza comparando el ID de Discord del usuario con el campo correspondiente en la base de datos.

Código 7: (Comprobación de propiedad de ticket)

```
1 if (ticket.discord_id !== interaction.user.id) {
2   return interaction.reply({ content: 'No tienes permiso para ver este
   ticket.', ephemeral: true });
3 }
```

6.3. Punto de Venta

El punto de venta (POS) de este proyecto es una aplicación web desarrollada con React para el frontend y Node.js para el backend, que permite a los usuarios realizar compras de manera sencilla y segura, integrando funcionalidades modernas como autenticación con Discord y pagos mediante PayPal.

Autenticación y Manejo de Sesión

El acceso al sistema requiere que el usuario inicie sesión utilizando su cuenta de Discord. El proceso de login se realiza mediante OAuth2, redirigiendo al usuario a la página de autorización de Discord. Una vez autorizado, el backend recibe un código temporal, lo intercambia por un token y obtiene los datos del usuario (ID, nombre, avatar, correo). Estos datos se almacenan en una cookie segura llamada **user**, que permite mantener la sesión activa y autenticar al usuario en cada solicitud.

Código 8: (Ejemplo de cookie de sesión)

```
1 {  
2   "id": "discord_id",  
3   "username": "nombre_usuario",  
4   "avatar": "url_avatar_discord",  
5   "email": "correo@ejemplo.com"  
6 }
```

Esta cookie es utilizada tanto por el frontend para mostrar el perfil del usuario como por el backend para identificar al usuario en operaciones como agregar productos al carrito, realizar compras y consultar tickets.

Gestión del Carrito y Productos

El usuario puede navegar por los productos disponibles, filtrarlos por categorías y agregarlos a su carrito. El carrito se almacena en la base de datos y está vinculado al usuario autenticado. El usuario puede modificar cantidades, eliminar productos y ver el total de su compra en tiempo real.

Integración con PayPal

Al proceder al pago, el sistema utiliza la API de PayPal para generar una orden de compra. El usuario es redirigido a la plataforma de PayPal para completar el pago de forma segura. Una vez realizado el pago, PayPal redirige al usuario de vuelta al sitio web, donde el backend verifica la transacción y registra la compra.

Código 9: (Flujo de pago con PayPal)

```
1 // Frontend: al completar el pago  
2 axios.post('/api/paypal/capture-order', {  
3   orderId,  
4   total  
5 }, { withCredentials: true })
```

Generación y Almacenamiento de Tickets

Tras una compra exitosa, el sistema genera un ticket de compra en la base de datos, asociado al usuario y con el detalle de los productos adquiridos. Cada ticket incluye una bandera booleana **notificado** con valor 1, indicando que está pendiente de notificación por parte del bot de Discord. El bot revisa periódicamente estos tickets y, al notificar al usuario, actualiza la bandera a 0.

Código 10: (Ejemplo de ticket de compra)

```
1 {  
2   "id": "ticket_id",  
3   "user_id": "discord_id",  
4   "products": [  
5     {  
6       "id": "producto_id",  
7       "name": "nombre_producto",  
8       "price": 100,  
9       "quantity": 2  
10    }  
11  ],  
12  "total": 200,  
13  "notificado": 1  
14 }
```


7. Resultados

7.1. Interfaces del Punto de Venta

7.1.1. Interfaz de Inicio de Sesión por Discord

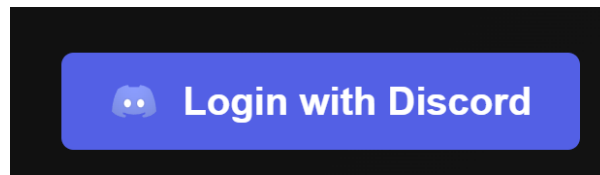


Figura 15: Botón de inicio de sesión con Discord

La figura 15 muestra el botón para realizar el inicio de sesión, este botón abrirá una nueva ventana con la interfaz para acceder con la cuenta de discord.

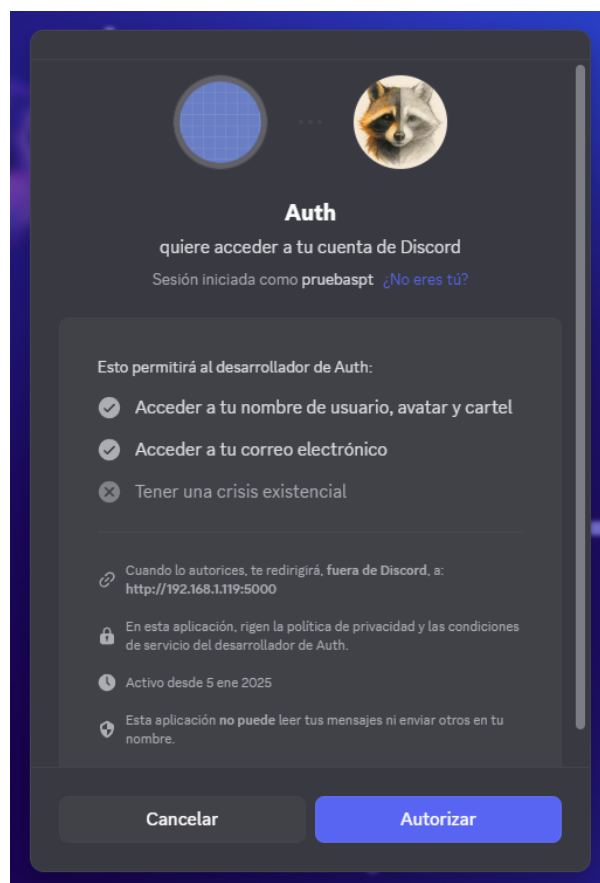


Figura 16: Interfaz OAuth2 de autenticación Discord

La figura 16 muestra la interfaz para la autenticación desde la API de Discord, donde el usuario debe autenticarse con su cuenta de Discord para acceder al sistema. Se utiliza el sistema OAuth2 de Discord para este propósito.

7.1.2. Interfaz de Página de Productos

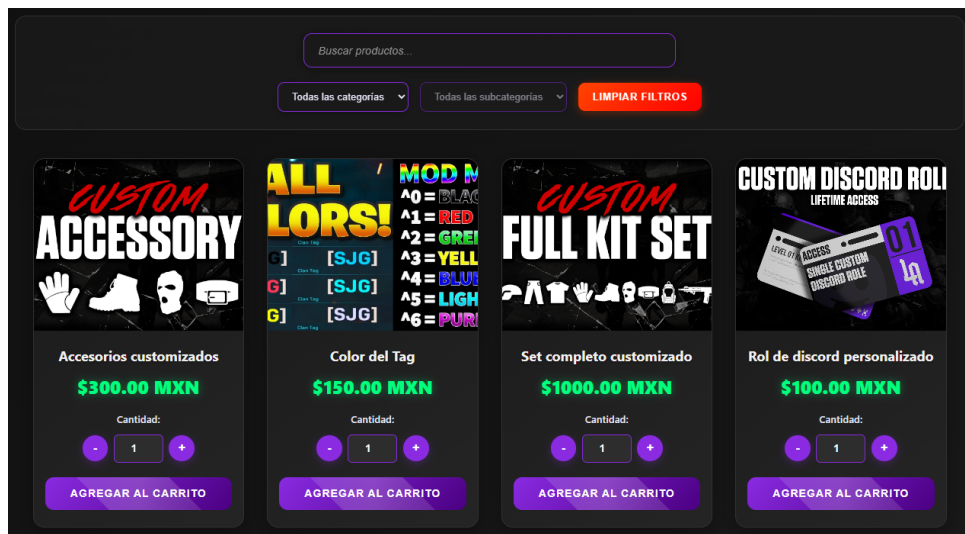


Figura 17: Vista de catálogo de productos

La figura 17 ilustra la página principal de la tienda, donde se muestran los productos disponibles. Los usuarios pueden filtrar y buscar productos, así como ver detalles de cada uno.

7.1.3. Interfaz del Carrito y el Procesamiento de Tickets

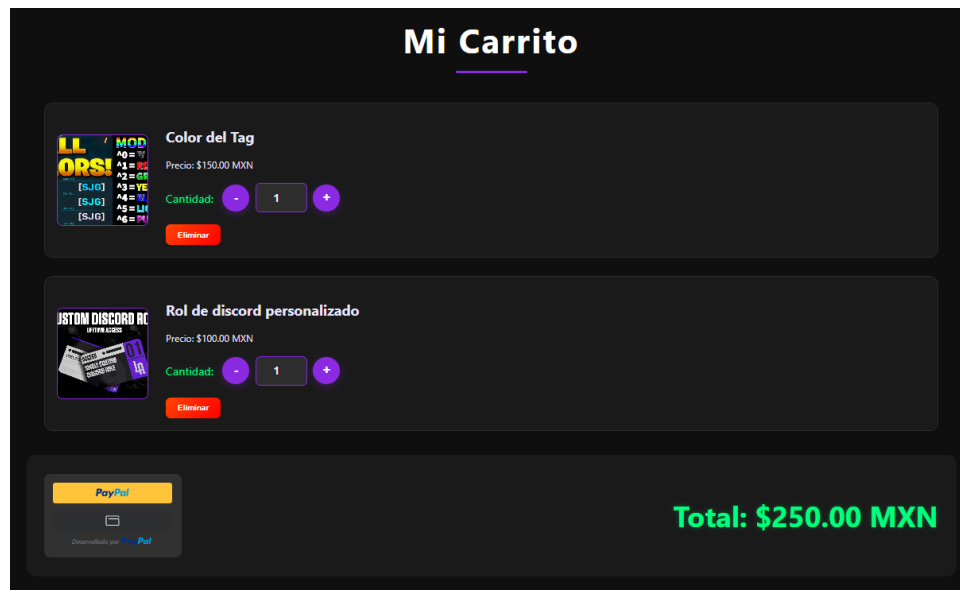


Figura 18: Vista del carrito de compras

En la figura 18, se puede observar el carrito de compras, donde el usuario puede revisar los productos seleccionados, modificar cantidades o eliminar artículos. También se muestra el total a pagar. Al proceder con la compra, se genera un ticket que queda registrado en el sistema.

7.2. funcionalidades del Bot

7.2.1. Creación de Comandos para Usuarios

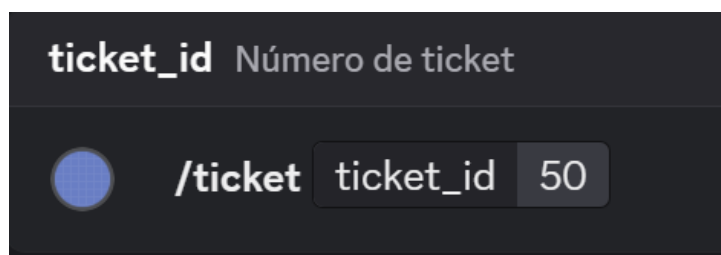


Figura 19: Comandos registrados del bot

La figura 19 muestra cómo se crean y gestionan los comandos del bot de Discord. Estos comandos permiten a los usuarios interactuar con el bot y realizar consultas sobre sus tickets o el estado de sus compras.

7.2.2. Comprobación de Tickets Pendientes

```
[NOTIFICACIONES] Tickets pendientes: 0
[DEBUG] Buscando ticket ID: 50 (Tipo: number)
[NOTIFICACIONES] Tickets pendientes: 0
[DEBUG] Buscando ticket ID: 51 (Tipo: number)
[DEBUG] Intentando crear canal para ticket ID: 51
[DEBUG] creando canal de ticket para ID: 51
[DEBUG] Canal creado exitosamente: ticket-51
[NOTIFICACIONES] Tickets pendientes: 0
[NOTIFICACIONES] Tickets pendientes: 0
```

Figura 20: Verificación de tickets pendientes

En la figura 20, se puede ver el proceso mediante el cual el bot verifica periódicamente si hay nuevos tickets pendientes de notificación. Esto asegura que los usuarios sean informados a tiempo sobre el estado de sus compras.

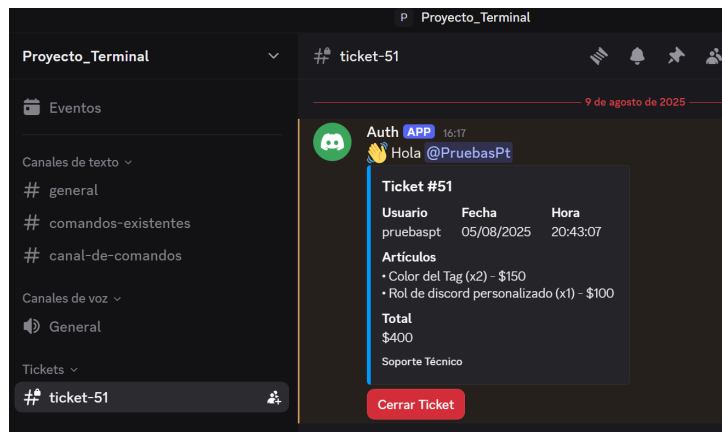


Figura 21: Notificación de ticket al usuario

En la figura 21, se muestra la notificación de el ticket de compra al usuario correspondiente.

7.2.3. Envío de Notificaciones por Canales Privados

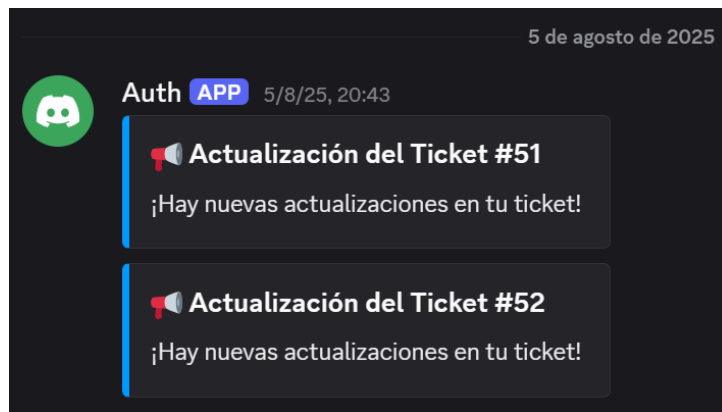


Figura 22: Mensaje privado de notificación

La figura 22 ilustra cómo el bot envía notificaciones directas a los usuarios a través de mensajes privados en Discord. Estas notificaciones pueden incluir información sobre el estado de un pedido, recordatorios de pago o promociones especiales.

8. Conclusiones

Durante el desarrollo de este proyecto se enfrentaron varios retos, especialmente el aprendizaje y uso de tecnologías nuevas como React para el frontend y la librería Discord.js para la integración del bot. La curva de aprendizaje fue significativa, ya que ambas tecnologías requieren comprender conceptos modernos de desarrollo web y asincronía en JavaScript. Sin embargo, la experiencia adquirida permitió implementar un sistema funcional que integra de manera eficiente la autenticación con Discord, la gestión de productos y carritos, el procesamiento de pagos con PayPal y la notificación automática de tickets mediante un bot.

El sistema desarrollado está orientado principalmente a pequeñas empresas o emprendimientos que ya cuenten con una comunidad activa en Discord. En estos entornos, la comunicación directa y rápida a través de la plataforma es una ventaja significativa, permitiendo notificaciones instantáneas y una interacción personalizada con los clientes. Sin embargo, se reconoce que el sistema no está diseñado para manejar un alto volumen de solicitudes como los que tendría una empresa grande. Para organizaciones de mayor tamaño, se recomienda utilizar canales de comunicación más formales o soluciones personalizadas que puedan escalar y soportar una mayor carga de usuarios y transacciones.

Un aspecto importante a considerar es la limitación impuesta por Discord en cuanto al envío de mensajes y notificaciones por parte de los bots. Discord establece límites estrictos en la cantidad de mensajes que un bot puede enviar en un periodo de tiempo determinado (rate limits). Por ejemplo, según la documentación oficial de Discord¹, los bots pueden enviar hasta 5 mensajes por segundo por canal, y existen límites globales y por usuario que, si se superan, pueden llevar a que el bot sea temporalmente bloqueado o incluso baneado. Esto implica que, en escenarios donde se requiera notificar a un gran número de usuarios simultáneamente, el sistema debe implementar mecanismos de control y espera para evitar superar estos límites y garantizar la continuidad del servicio.

¹Discord Developer Portal, Rate Limits. Disponible en: <https://discord.com/developers/docs/topics/rate-limits>

9. Referencias

Referencias

- [1] PayPal. Bienvenido a PayPal. Disponible en: <https://www.paypal.com/mx/webapps/mpp/welcome-to-paypal>. Accedido: 21-jun-2025.
- [2] MEE6. MEE6 - El bot de Discord más completo. Disponible en: <https://mee6.xyz/es>. Accedido: 21-jun-2025.
- [3] R. Teja Rubio. Proyecto Tienda Virtual: Uniformes Escolares. UOC, 2016. Disponible en: <https://openaccess.uoc.edu/handle/10609/49345>
- [4] Facebook. Marketplace de Facebook. Disponible en: <https://www.facebook.com/marketplace/>. Accedido: 21-jun-2025.
- [5] AliExpress. Disponible en: <https://es.aliexpress.com/>. Accedido: 21-jun-2025.